

Ejercicios adicionales

1 Ejercicio 1

Ordene las siguientes funciones: $\sqrt{n}, n, 3^n, n^2, cte, 2^n, \log_2^2(n), \log_3(n), \log_2(n)$ segun su velocidad de crecimiento.

Ordenando de menor a mayor:

$$cte < \log_2(n) < \log_2^2(n) < \sqrt{n} < n < n^2 < 2^n < 3^n$$

2 Ejercicio 2

Indique de qué orden es el siguiente fragmento de código.

```
for (int j = 4; j < n; j=j+2) {  
    val = 0;  
    for (int i = 0; i < j; ++i) {  
        val = val + i * j;  
        for (int k = 0; k < n; ++k){  
            val++;  
        }  
    }  
}
```

(a) $O(n \log n)$ (b) $O(n^2)$ (c) $O(n^2 \log n)$ (e) $O(n^3)$

Indices:

- for (int j = 4; j < n; j=j+2)

1. $j = 4 = 2 \cdot 2$
2. $j = 2 \cdot 2 + 2 = 3 \cdot 3$
3. $j = 3 \cdot 2 + 2 = 4 \cdot 3$
- $\vdots$

Paso general: $h \cdot j = 2(h + 1)$ Despejando:

$$\begin{aligned} n &= 2(h + 1) \\ \frac{n}{2} &= h + 1 \\ \frac{n}{2} - 1 &= h \end{aligned}$$

Primera sumatoria: $\sum_{j=1}^{\frac{n}{2}-1}$

- for (int i = 0; i < j; ++i)

Este índice se ejecuta dependiendo del valor de la variable j de la sumatoria externa.

- Primera iteración: Se ejecuta 4 veces, desde i=0 hasta 3.
 - Segunda iteración: Se ejecuta 6 veces, desde i=0 hasta 5.
 - Tercera iteración: Se ejecuta 8 veces, desde i=0 hasta 7.
- Y así...

Cumple un patrón de $2j + 1$.

Segunda sumatoria: $\sum_{i=0}^{2j+1}$

- for (int k = 0; k < n; ++k) Este índice siempre va desde $k = 0$ hasta n .

Sumatoria: $\sum_{k=0}^n$

$$\begin{aligned} T(n) &= \sum_{j=1}^{\frac{n}{2}-1} [c_1 + \sum_{i=0}^{2j+1} (c_2 + \sum_{k=0}^n c_3)] \\ &= \sum_{j=1}^{\frac{n}{2}-1} [c_1 + \sum_{i=0}^{2j+1} [c_2 + c_3(n - 1)]] \\ &= \sum_{j=1}^{\frac{n}{2}-1} [c_1 + (c_2 + c_3n - c_3)(2j + 1 - 1)] \\ &= \sum_{j=1}^{\frac{n}{2}-1} c_1 + (c_2 + c_3n - c_3) \sum_{j=1}^{\frac{n}{2}-1} j \\ &= c_1 \left(\frac{n}{2} - 1 - 1 + 1 \right) + 2(c_2 - c_3) \sum_{j=1}^{\frac{n}{2}-1} j + 2c_3n \sum_{j=1}^{\frac{n}{2}-1} j \\ &= \frac{c_1}{2}n - c_1 + 2(c_2 - c_3) \left(\frac{(\frac{n}{2}-1)\frac{n}{2}}{2} \right) + 2c_3n \left(\frac{(\frac{n}{2}-1)\frac{n}{2}}{2} \right) \\ &= \frac{c_1}{2}n - c_1 + (c_2 - c_3) \left(\frac{n^2}{4} - \frac{n}{2} \right) + c_3n \left(\frac{n^2}{4} - \frac{n}{2} \right) \\ &= -c_1 + \frac{c_1}{2}n - \frac{c_2}{2}n - \frac{c_3}{2}n + \frac{c_2}{4}n^2 - \frac{c_3}{4}n^2 + \frac{c_3}{2}n^2 + \frac{c_3}{4}n^3 \\ &= -c_1 + n \left(\frac{c_1}{2} - \frac{c_2}{2} - \frac{c_3}{2} \right) + n^2 \left(\frac{c_2}{4} - \frac{c_3}{4} + \frac{c_3}{2} \right) + \frac{c_3}{4}n^3 \end{aligned}$$

Opción correcta: $O(n^3)$

3 Ejercicio 3

Suponga que dispone de un algoritmo A, que resuelve un problema de tamaño n , y su función de tiempo de ejecución es $T(n) = n * \log(n)$. Este algoritmo se ejecuta en una computadora que procesa 10.000 operaciones por segundo. Determine el tiempo que requerirá el algoritmo para resolver un problema de tamaño $n=1024$.

Calculamos la cantidad de operaciones para $T(n)$.

$$\begin{aligned} T(1024) &= 1024 \cdot \log_2(1024) \\ &= 10240 \end{aligned}$$

Ahora hallamos el tiempo que requiere el algoritmo.

10000 operaciones	_____	1 segundo
10240 operaciones	_____	$x = 1,024$ segundos

4 Ejercicio 4

¿Cuál de las siguientes sentencias es correcta, según la definición vista en clase?

- a. n^2 es $O(n^2)$
- b. n^2 es $O(n^3)$
- c. n^2 es $O(n^2 \log n)$
- d. Opciones a y b
- e. Opciones a, b y c
- f. Ninguna de las anteriores

5 Ejercicio 5

Dado el siguiente algoritmo

```
void ejercicio5 (int n) {
    if (n >= 2) {
        2 * ejercicio5 (n/2);
        n = n/2;
        ejercicio5 (n/2);
    }
}
```

1. Indique el $T(n)$ para $n \geq 2$.

- a. $T(n) = d + 3 \cdot T(\frac{n}{2})$
- b. $T(n) = d + 2 \cdot T(\frac{n}{2}) + T(\frac{n}{4})$
- c. $T(n) = d + T(\frac{n}{2}) + T(\frac{n}{4})$
- d. $T(n) = d + T(\frac{n}{2}) + T(\frac{n}{2})$
- e. $T(n) = d + T(\frac{n}{2}) + T(\frac{n}{2}) + T(\frac{n}{4})$

6 Ejercicio 6

Considere el siguiente fragmento de código:

```
int count = 0; int n = a.length;
for (int i = 0; i < n; i+=n/2) {
    for (int j = 0; j < n; j++) {
        a[j]++;
    }
}
```

Este algoritmo se ejecuta en una computadora que procesa 100.000 operaciones por cada segundo. Determine el tiempo **aproximado** que requerirá el algoritmo para resolver un problema de tamaño $n=1000$.

- a. 0,01 seg
- b. 0,1 seg
- c. 1 seg
- d. Ninguna de las opciones anteriores.

Calculamos la función del tiempo de ejecución del algoritmo.

- La primera línea, las asignaciones, son operaciones constantes.
- El primer for se ejecuta 2 veces.

- 1. $i = 0$
- 2. $i = \frac{n}{2}$
- 3. $i = \frac{n}{2} + \frac{n}{2} = 2\frac{n}{2} = n$. Ya no se ejecuta.

- El segundo for se ejecuta desde $j=0$ hasta $n-1$, porque j es estrictamente menor que n .

$$\begin{aligned}
T(n) &= c_1 + 2 \sum_{j=0}^{n-1} c_2 \\
&= c_1 + 2[c_2(n - 1 + 0 + 1)] \\
&= c_1 + 2c_2n
\end{aligned}$$

La función $T(n)$ es de orden lineal. Como es el tiempo **aproximado**, tomamos ese orden.

Llamamos $p(n)$ a la función aproximada. $p(1000) = 1000$ Ahora calculamos el tiempo requerido

$$\begin{array}{rcl}
100000 \text{ operaciones} & \text{-----} & 1 \text{ segundo} \\
1000 \text{ operaciones} & \text{-----} & x = 0,01 \text{ segundos}
\end{array}$$

Por lo tanto, el algoritmo requerirá 0,01 segundos para resolver un problema de ese tamaño.

7 Ejercicio 7

Expresar la función $T(n)$ del siguiente segmento de código:

```

public static void ejercicio (int n) {
    int x = 0;
    int j = 1;
    while ( j <= n ) {
        for ( int i = n*n ; i >=1 ; i = i - 3 )
            x = x + 1 ;
        j = j * 2 ;
    }
}

```

- $T(n) = (1/3) * n^2 + \log_2 n$
- $T(n) = n^2 + (1/3) * \log_2(n)$
- $T(n) = (1/3) * \log_2(n)$
- $T(n) = (1/3) * n^2 * \log_2(n) + \log_2(n)$

En este ejercicio no me dio exactamente el mismo así que marqué el más parecido.

$$\begin{aligned}
T(n) &= \sum_{j=1}^{1+\log_2 n} \left[\sum_{i=0}^{\frac{n^2+1}{3}} 1 \right] \\
&= \sum_{j=1}^{1+\log_2 n} \left[\frac{n^2+1}{3} - 1 + 1 \right] \\
&= \sum_{j=1}^{1+\log_2 n} \frac{n^2+1}{3} \\
&= \sum_{j=1}^{1+\log_2 n} \frac{n^2}{3} + \sum_{j=1}^{1+\log_2 n} \frac{1}{3} \\
&= \frac{n^2}{3} (\log_2(n) + 1 - 1 + 1) + \frac{1}{3} (\log_2(n) + 1 - 1 + 1) \\
&= \frac{n^2}{3} \log_2(n) + \frac{1}{3} \log_2(n) + 1
\end{aligned}$$